



南京大學
NANJING UNIVERSITY

四、中断系统

主讲教师：闫浩
机器人与自动化学院



目录

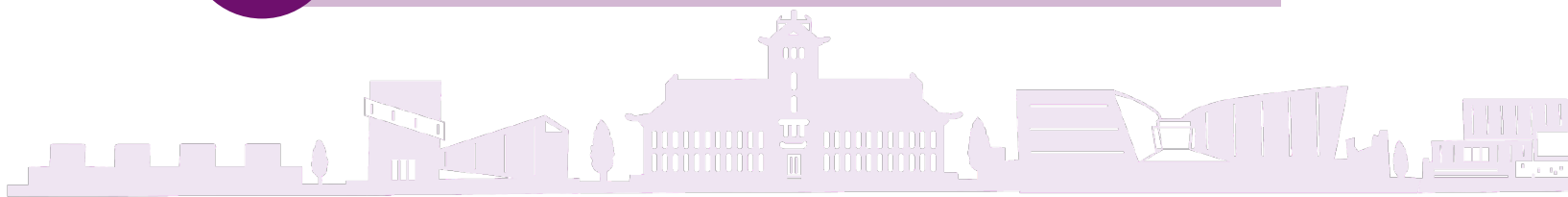
CONTENTS

01 中断的基本概念

02 STM32中断系统

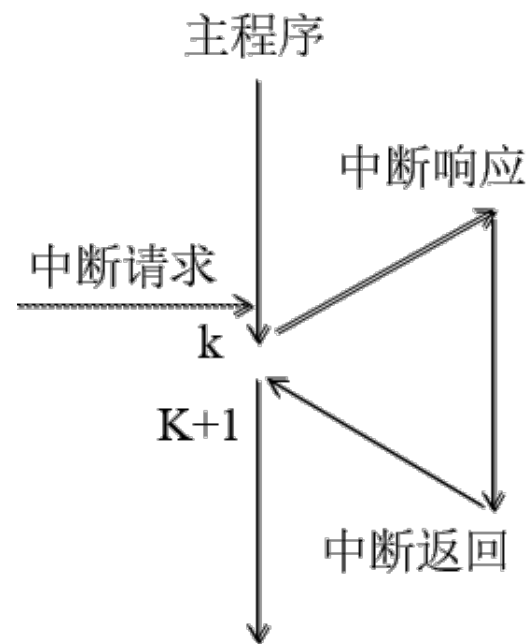
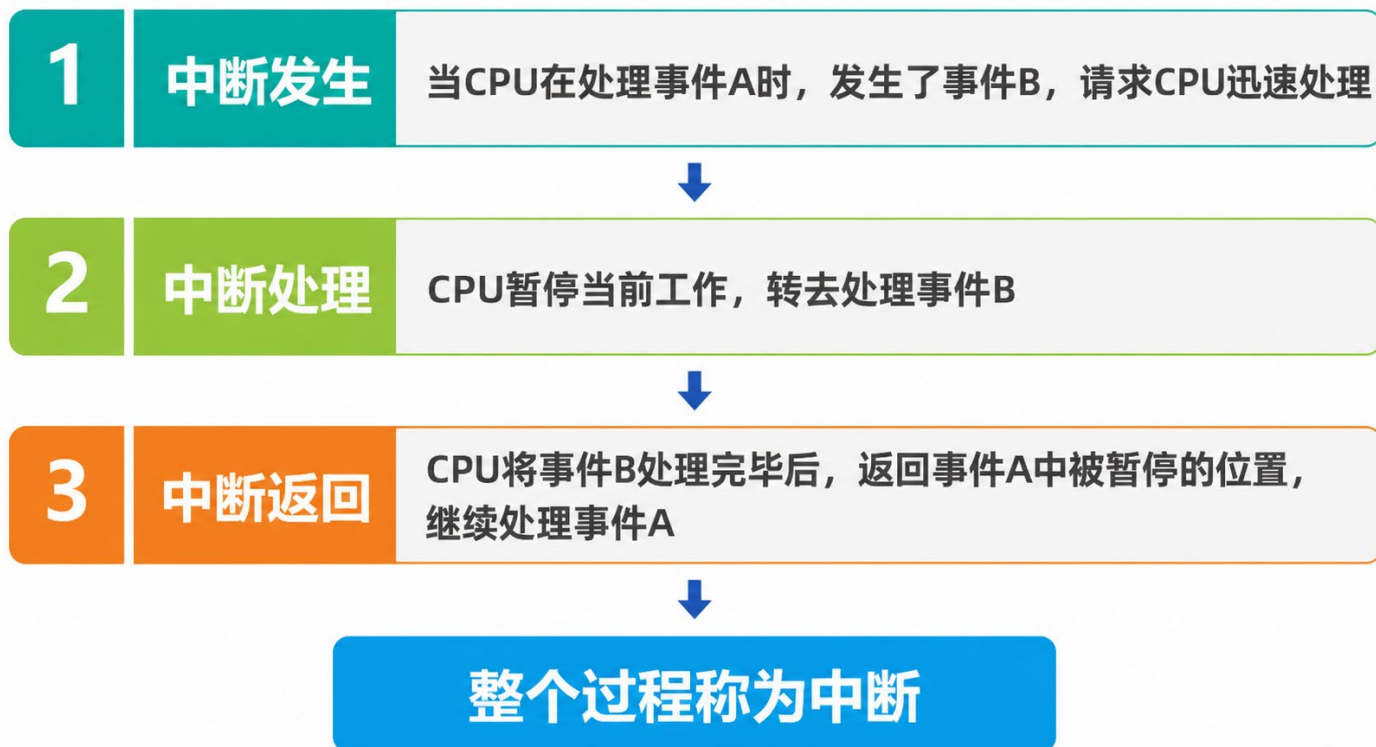
03 STM32中断系统库函数

04 应用实例



01 中断的基本概念

在计算机执行程序的过程中，CPU暂时中止其正在执行的程序，转去执行请求中断的那个外设或事件的服务程序，等处理完后再返回执行原来中止的程序，叫作中断。



1. 中断源

中断源是指能引发中断的事件。通常，中断源都与外设有关。计算机系统中，常见的中断源有**按键、定时器溢出、串口收到数据**等，与此相关的外设有**键盘、定时器和串口**等。

2. 中断请求标志位

每个中断源都有它对应的中断标志位，一旦该中断发生，它的中断标志位就会被置位。如果中断标志位被清除，那么它所对应的中断便不会再被响应。所以，**一般在中断服务程序最后要将对应的中断标志位清零**，否则将始终响应该中断，不断执行该中断服务程序。

3. 中断屏蔽

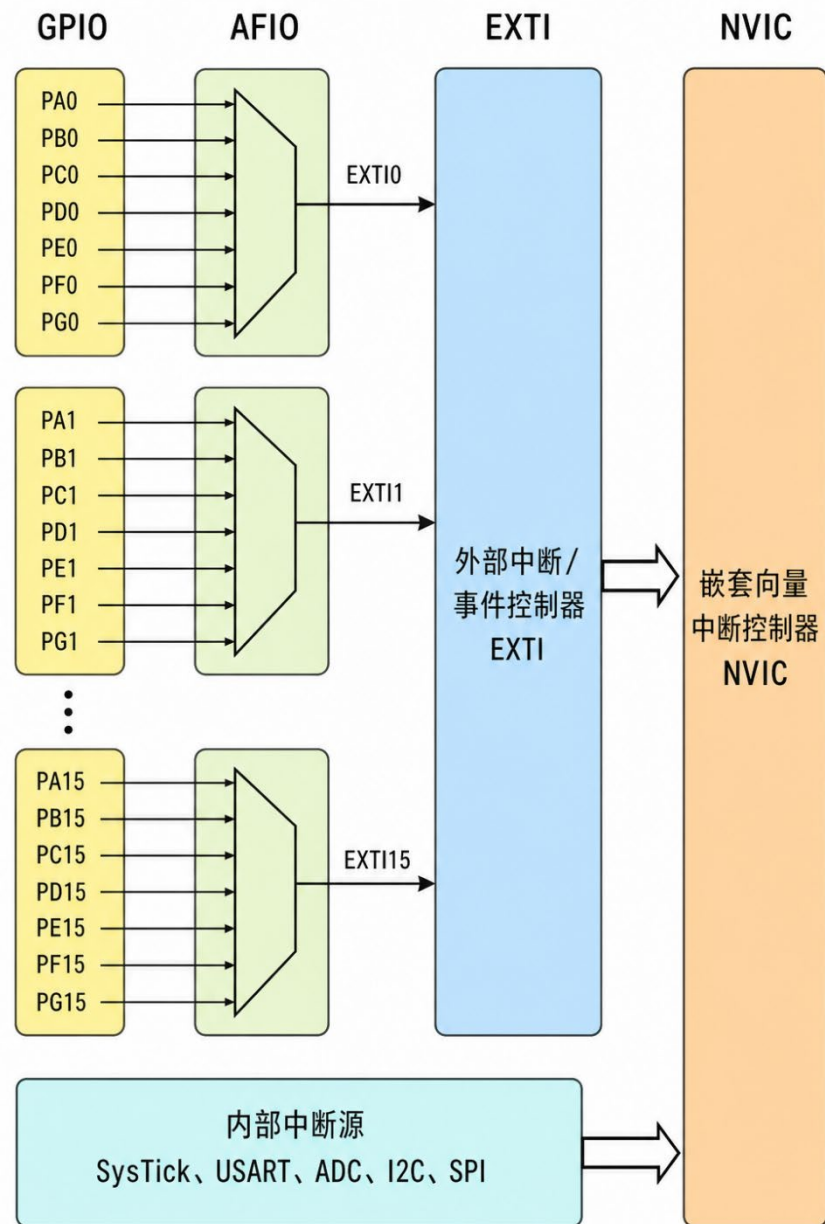
可通过设置相应的中断屏蔽位，禁止CPU响应某个中断，从而实现某个中断。“中断允许总控制位”和“中断允许控制位”任一个关闭，该中断无法响应。分为“可屏蔽中断”和“不可屏蔽中断”

中断的分类及发生路径

中断的分类

内部中断（也称异常）

外部中断（也称中断）



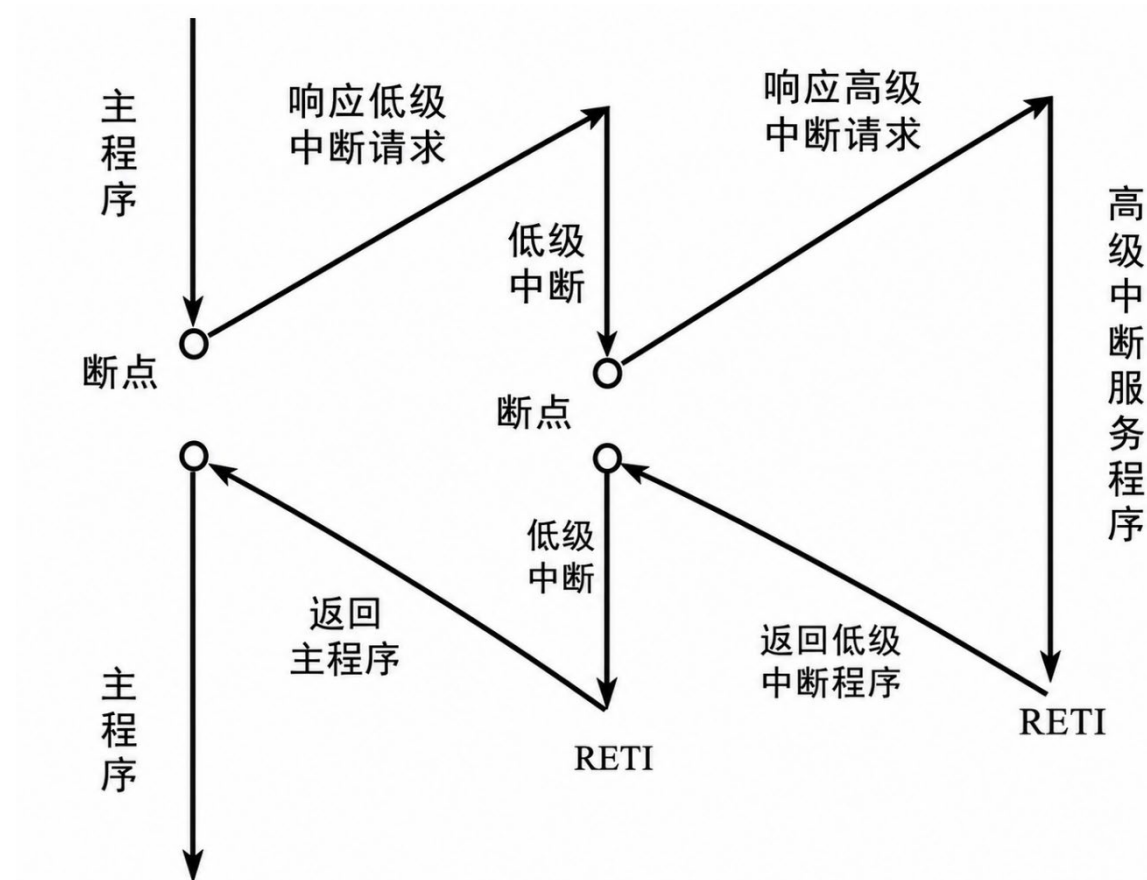
中断类型举例

优先级	优先级类型	名称	说明	地址
-	-	-	保留（实际存的是 MSP 地址）	0x0000 0000
-3	固定	Reset	复位	0x0000 0004
-2	固定	NMI	不可屏蔽中断。RCC 时钟安全系统 (CSS) 连接到 NMI 向量	0x0000 0008
-1	固定	HardFault	所有类型的错误	0x0000 000C
0	可编程	MemManage	存储器管理	0x0000 0010
1	可编程	BusFault	预取指失败，存储器访问失败	0x0000 0014
2	可编程	UsageFault	未定义的指令或非法状态	0x0000 0018
-	-	-	保留	0x0000 001C-0x0000 002B
3	可编程	SVCall	通过 SWI 指令调用的系统服务	0x0000 002C
4	可编程	Debug Monitor	调试监控器	0x0000 0030
-	-	-	保留	0x0000 0034
5	可编程	PendSV	可挂起的系统服务	0x0000 0038
6	可编程	SysTick	系统嘀嗒定时器	0x0000 003C

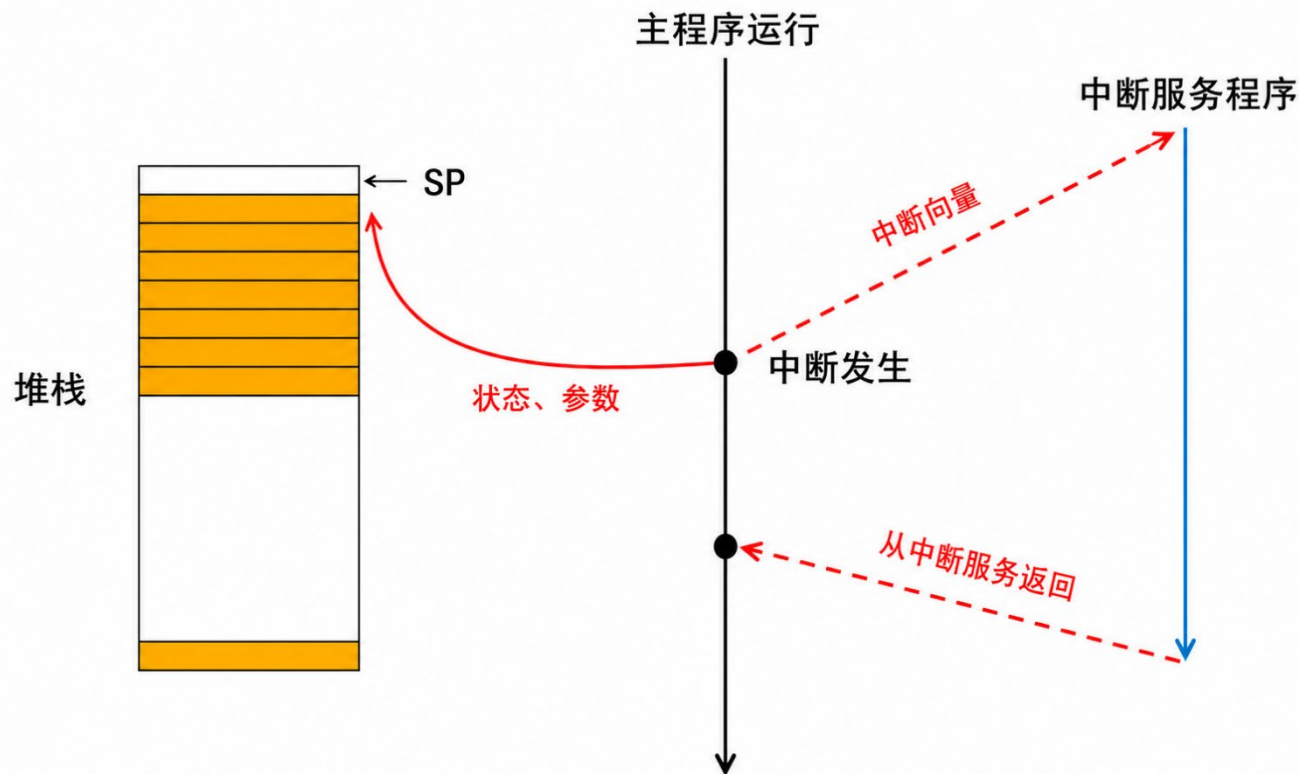
位置	优先级	优先级类型	名称	说明	地址
0	7	可设置	WWDG	窗口看门狗中断	0x0000 0040
1	8	可设置	PVD	连接到 EXTI 线的可编程电压检测 (PVD)中断	0x0000 0044
2	9	可设置	TAMP_STAMP	连接到 EXTI 线的入侵和时间戳中断	0x0000 0048
3	10	可设置	RTC_WKUP	连接到 EXTI 线的 RTC 唤醒中断	0x0000 004C
4	11	可设置	FLASH Flash	全局中断	0x0000 0050
5	12	可设置	RCC RCC	全局中断	0x0000 0054
6	13	可设置	EXTI0 EXTI	线 0 中断	0x0000 0058
7	14	可设置	EXTI1 EXTI	线 1 中断	0x0000 005C
8	15	可设置	EXTI2 EXTI	线 2 中断	0x0000 0060
中间其他中断省略					
84	91	可编程	SPI4	SPI4 全局中断	0x0000 0190
85	92	可编程	SPI5	SPI5 全局中断	0x0000 0194
86	93	可编程	SPI6	SPI6 全局中断	0x0000 0198
87	94	可编程	SAI1	SAI1 全局中断	0x0000 019C
88	95	可编程	LTDC	LTDC 全局中断	0x0000 01A0
89	96	可编程	LTDC_ER	LTDC_ER 全局中断	0x0000 01A4

中断优先级与中断嵌套

当一个中断程序正在运行时，又有新的更高优先级中断源申请中断，CPU就会再次暂停当前中断程序，然后去处理新的中断程序，处理完成后依次进行返回。



中断服务程序 (Interrupt Service Routine, ISR) 又叫中断处理函数，中断服务例程等。是在中断发生时，由CPU自动执行的一段特殊的软件代码。每一个ISR都有一个固定的入口地址，称为中断向量，这些地址存放在中断向量表中。



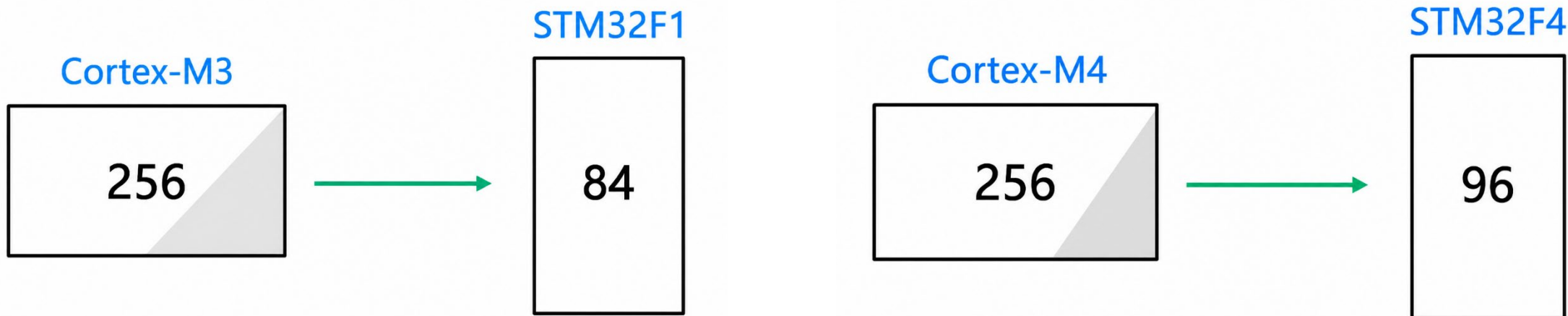
- **实时控制：**在确定时间内对相应事件作出实时响应，如温度监控
- **故障处理：**检测到故障，需要第一时间处理，如:电梯门夹人了
- **提高效率：**CPU无需持续检查所有设备状态，提高CPU工作效率
- **分时操作：**有效地在多个任务之间进行切换，实现分时操作

02 STM32中断系统

Cortex-M3/M4内核支持256个中断，其中包含了16个内核中断和240个外部中断，具有256级的可以编程设置的中断优先级。

STM32F1只是用了M3内核的一部分，它只含有84个中断源，其中包括16个内核中断和68个外部可屏蔽中断。

STM32F4只是用了M4内核的一部分，它只含有96个中断源，其中包括10个内核中断和86个外部可屏蔽中断。

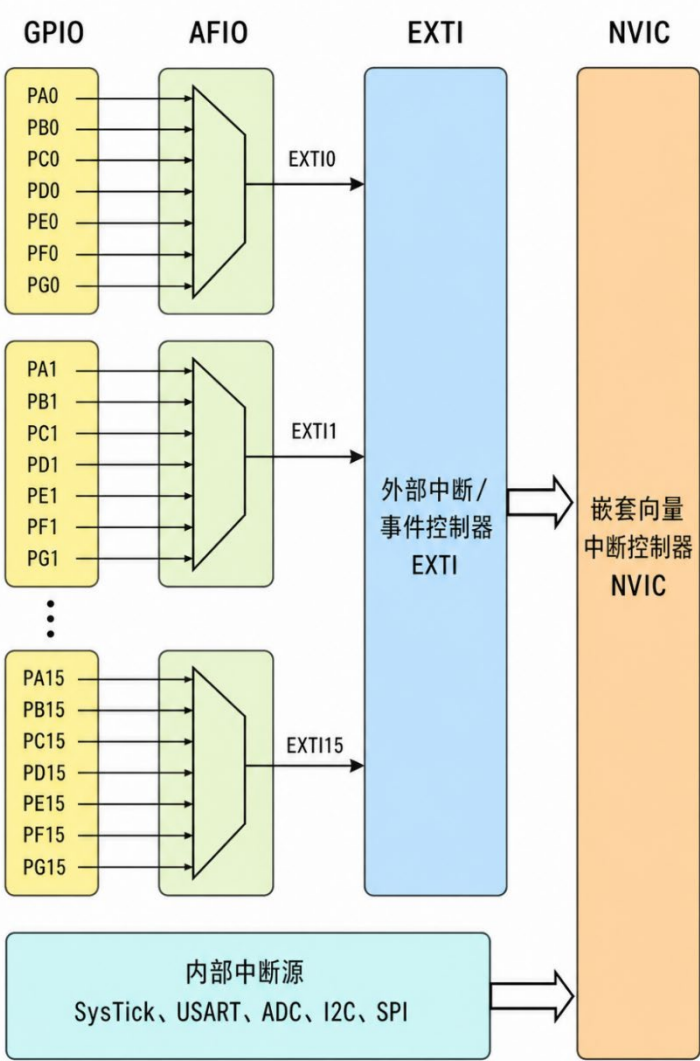


NVIC中断管理方法

1、中断优先级分组

对STM32中断进行分组为0~4。同时，对每个中断设置一个抢占优先级和一个响应优先级值。
分组配置是在寄存器SCB_AIRCR(Application interrupt and reset control register:应用中断和复位控制寄存器)中配置：

组	AIRCR[10: 8]	IP bit[7: 4]分配情况	分配结果
0	111	0:4	0位抢占优先级，4位响应优先级
1	110	1:3	1位抢占优先级，3位响应优先级
2	101	2:2	2位抢占优先级，2位响应优先级
3	100	3:1	3位抢占优先级，1位响应优先级
4	011	4:0	4位抢占优先级，0位响应优先级



NVIC中断管理方法

优先级分组方式所对应的抢占式优先级和响应优先级寄存器位数和所表示的优先级数如图所示



STM32优先级位数和级数分配图

在 STM32中，中断优先级数值越小，优先级越高，0 是最高优先级

NVIC中断管理方法

2、中断优先级管理方法

- 1) 抢占优先级较高的中断是可以打断正在进行的抢占优先级较低的中断;
- 2) 抢占优先级相同的中断, 响应优先级高的**不可以**打断响应优先级低的中断;
- 3) 抢占优先级相同的中断, 当两个中断同时发生的情况下, 哪个响应优先级高, 哪个先执行;
- 4) 如果两个中断的抢占优先级和响应优先级**都是一样**的话, 则看哪个中断先发生就先执行。
- 5) 如果两个中断的抢占优先级和响应优先级**都是一样**的话, 且同时请求, 则根据中断表中的排位顺序决定。

抢占式优先级 > 响应优先级 > 中断表中的排位顺序

NVIC中断管理方法

3、举例

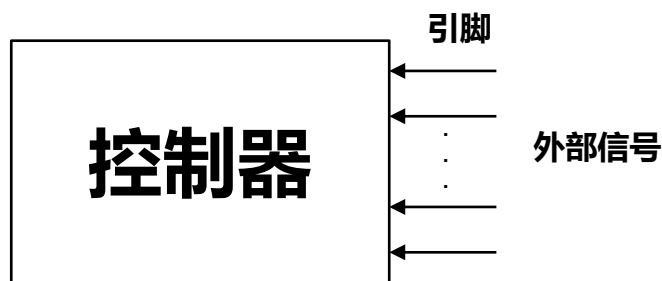
假定设置中断优先级组为2，然后设置：

--中断3(RTC中断)的抢占优先级为2，响应优先级为1。

--中断6（外部中断0）的抢占优先级为3，响应优先级为0。

--中断7（外部中断1）的抢占优先级为2，响应优先级为0。

那么这3个中断的优先级顺序为：中断7>中断3>中断6。



通过外部信号触发一个中断，从而启动一个对应于外部信号的服务。

- 1、STM32的每个IO都可以作为外部中断输入。
- 2、STM32的中断控制器支持22个外部中断/事件请求：

EXTI线0~15：对应外部IO口的输入中断。

EXTI线16：连接到PVD输出。

EXTI线17：连接到RTC闹钟事件。

EXTI线18：连接到USB OTG FS唤醒事件。

EXTI线19：连接到以太网唤醒事件。

EXTI线20：连接到USB OTG HS(在FS中配置)唤醒事件。

EXTI线21：连接到RTC入侵和时间戳事件。

EXTI线22：连接到RTC唤醒事件。

- 3、每个外部中断线可以独立的配置触发方式（上升沿，下降沿或者双边沿触发），触发/屏蔽，专用的状态位。

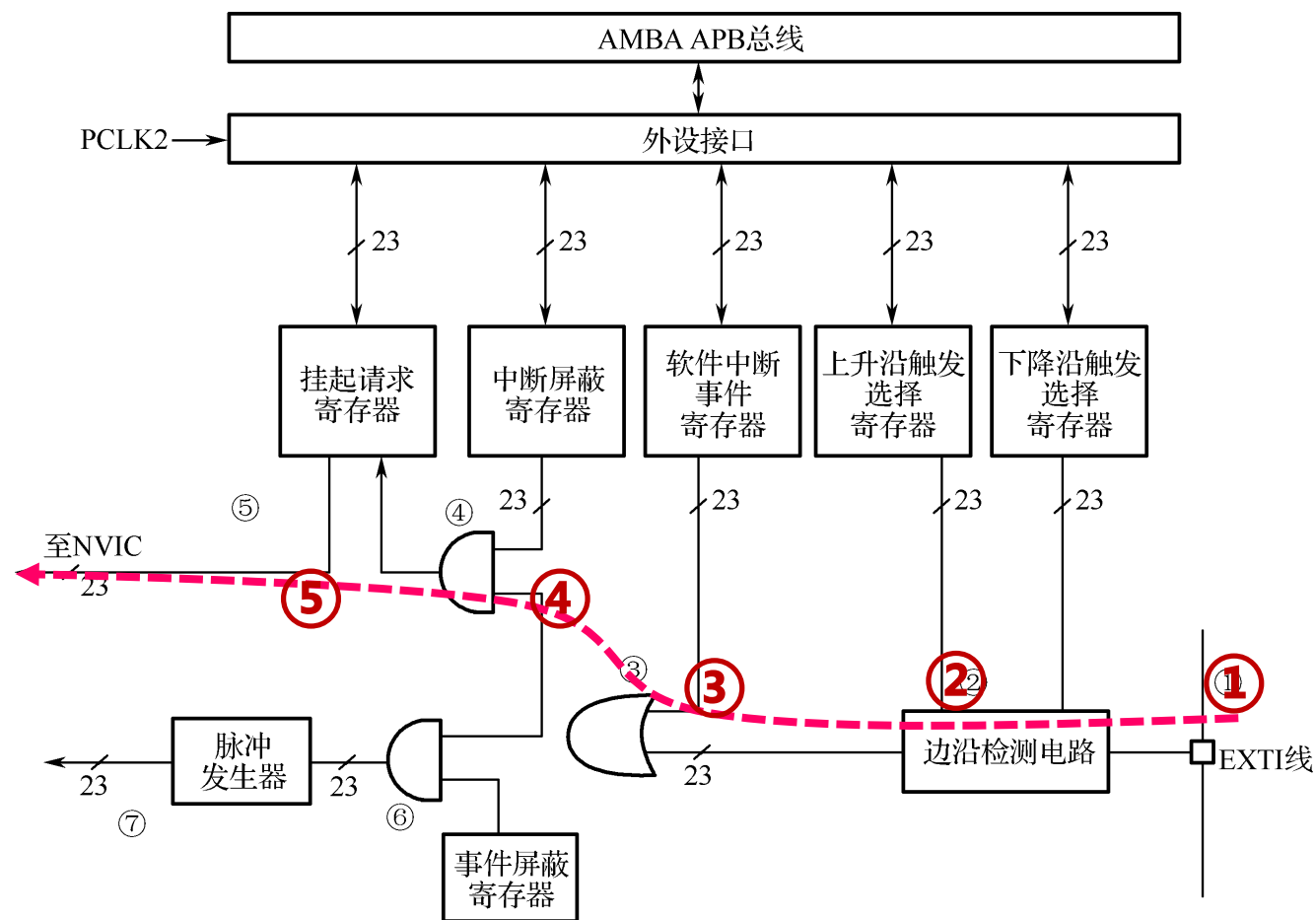
STM32外部中断/事件控制器 (EXTI)

结构

1. 中断请求

请求信号通过图中的①②③④⑤的路径向NVIC产生中断请求。

对于每个中断线，我们可以设置相应的触发方式（上升沿触发，下降沿触发，边沿触发）以及使能。

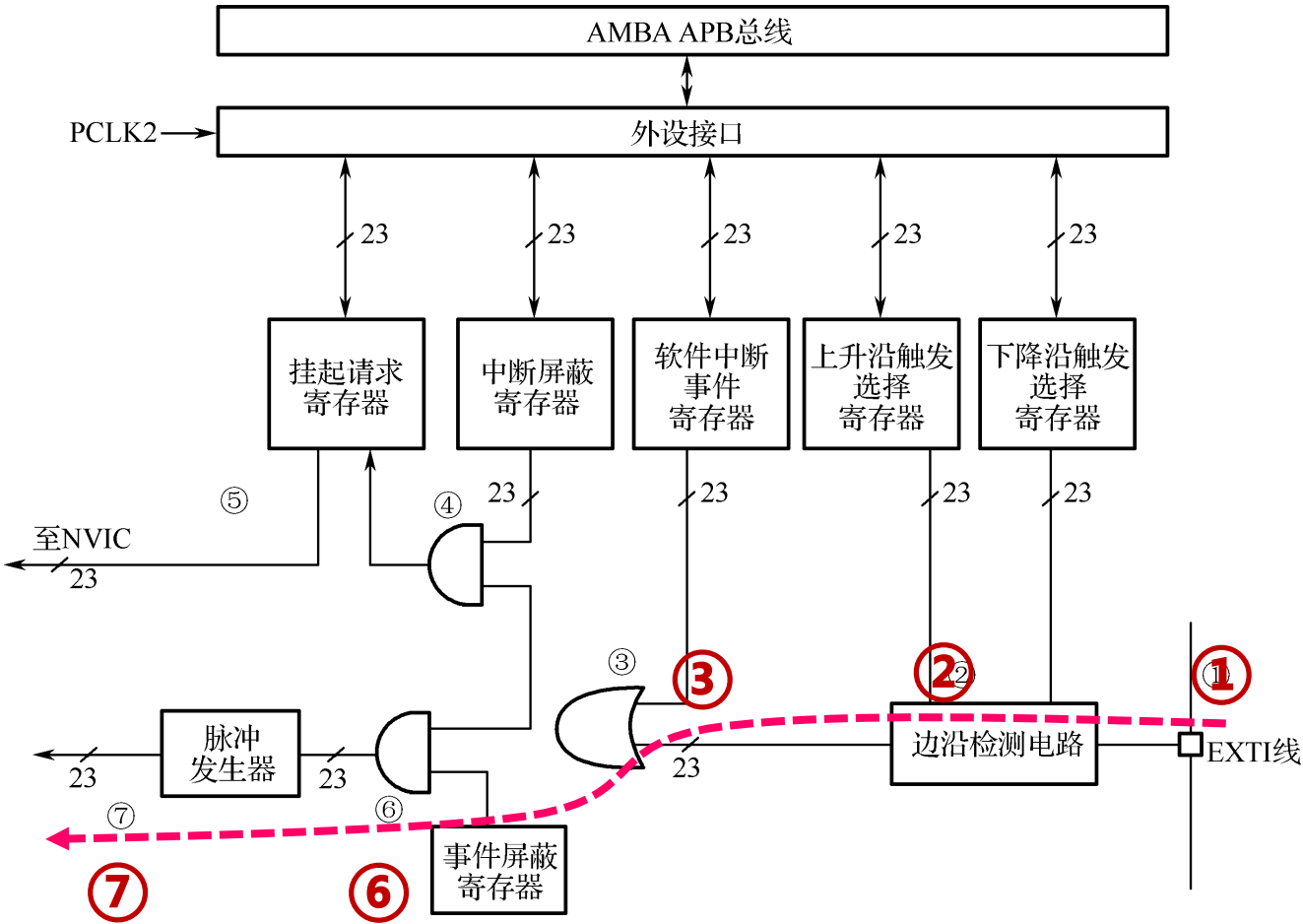


EXTI的结构图

结构

2、触发事件

请求信号通过图中的
①②③⑥⑦的路径产生触发事件。



EXTI的结构图

GPIO相关EXTI线

GPIOx.0映射到EXTI0

GPIOx.1映射到EXTI1

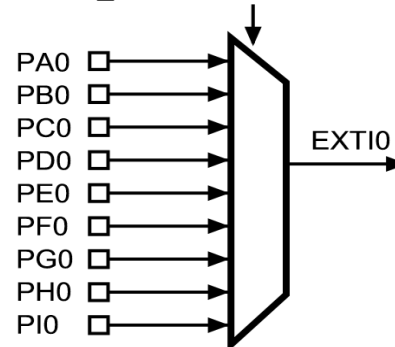
...

GPIOx.15映射到EXTI15

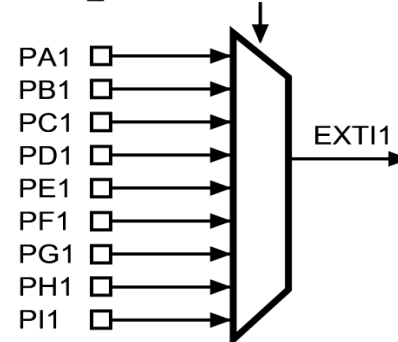
另外七根 EXTI 线连接方式如下:

- EXTI 线 16 连接到 PVD 输出
- EXTI 线 17 连接到 RTC 闹钟事件
- EXTI 线 18 连接到 USB OTG FS 唤醒事件
- EXTI 线 19 连接到以太网唤醒事件
- EXTI 线 20 连接到 USB OTG HS (在 FS 中配置) 唤醒事件
- EXTI 线 21 连接到 RTC 入侵和时间戳事件
- EXTI 线 22 连接到 RTC 唤醒事件

SYSCFG_EXTICR1 寄存器中的 EXTI0[3:0] 位

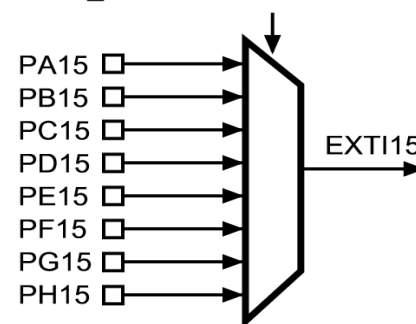


SYSCFG_EXTICR1 寄存器中的 EXTI1[3:0] 位



⋮

SYSCFG_EXTICR4 寄存器中的 EXTI15[3:0] 位



外部中断服务

- 1、16个中断线**没有**对应16个独立的中断服务；
- 2、IO口外部中断在中断向量表中只分配了7个中断向量，也就是只能使用7个中断服务函数

位置	优先级	优先级类型	名称	说明	地址
6	13	可设置	EXTI0	EXTI 线 0 中断	0x0000 0058
7	14	可设置	EXTI1	EXTI 线 1 中断	0x0000 005C
8	15	可设置	EXTI2	EXTI 线 2 中断	0x0000 0060
9	16	可设置	EXTI3	EXTI 线 3 中断	0x0000 0064
10	17	可设置	EXTI4	EXTI 线 4 中断	0x0000 0068
23	30	可设置	EXTI9_5	EXTI 线 [9:5] 中断	0x0000 009C
40	47	可设置	EXTI15_10	EXTI 线 [15:10] 中断	0x0000 00E0

- 3、**外部中断线5~9分配一个中断向量，共用一个服务函数；**
外部中断线10~15分配一个中断向量，共用一个中断服务函数。

外部中断服务

4、中断服务函数列表：

```
EXTI0_IRQHandler  
EXTI1_IRQHandler  
EXTI2_IRQHandler  
EXTI3_IRQHandler  
EXTI4_IRQHandler  
EXTI9_5_IRQHandler  
EXTI15_10_IRQHandler
```

03 STM32中断系统库函数

NVIC库函数

函数名	描述
NVIC_Delnit	将外设NVIC寄存器重设为默认值
NVIC_SCBDelnit	将外设SCB寄存器重设为默认值
NVIC_PriorityGroupConfig	设置优先级分组：抢占优先级和响应优先级
NVIC_Init	根据NVIC_InitStruct中指定的参数初始化外设NVIC寄存器
NVIC_StructInit	把NVIC_InitStruct中的每一个参数按默认值填入
NVIC_SETPRIMASK	使能PRIMASK优先级：提升执行优先级至0
NVIC_RESETPRIMASK	失能PRIMASK优先级
NVIC_SETFAULTMASK	使能FAULTMASK优先级：提升执行优先级至-1
NVIC_RESETFAULTMASK	失能FAULTMASK优先级
NVIC_BASEPRICONFIG	改变执行优先级从N（最低可设置优先级）提升至1
NVIC_GetBASEPRI	返回BASEPRI屏蔽值
NVIC_GetCurrentPendingIRQChannel	返回当前待处理IRQ标识符
NVIC_GetIRQChannelPendingBitStatus	检查指定的IRQ通道待处理位设置与否
NVIC_SetIRQChannelPendingBit	设置指定的IRQ通道待处理位
NVIC_ClearIRQChannelPendingBit	清除指定的IRQ通道待处理位
NVIC_GetCurrentActiveHandler	返回当前活动Handler（IRQ通道和系统Handler）的标识符
NVIC_GetIRQChannelActiveBitStatus	检查指定的IRQ通道活动位设置与否
NVIC_GetCUID	返回ID号码，Cortex-M3内核的版本号和实现细节
NVIC_SetVectorTable	设置向量表的位置和偏移
NVIC_GenerateSystemReset	产生一个系统复位
NVIC_GenerateCoreReset	产生一个内核（内核+NVIC）复位
NVIC_SystemLPConfig	选择系统进入低功耗模式的条件
NVIC_SystemHandlerConfig	使能或者失能指定的系统Handler
NVIC_SystemHandlerPriorityConfig	设置指定的系统Handler优先级
NVIC_CetSystemHandlerPendingBitStatus	检查指定的系统Handler待处理位设置与否
NVIC_SetSystemHandlerPendingBit	设置系统Handler待处理位
NVIC_ClearSystemHandlerPendingBit	清除系统Handler待处理位
NVIC_GetSystemHandlerActiveBitStatus	检查系统Handler活动位设置与否
NVIC_GetFaultHandlerSources	返回表示出错的系统Handler源
NVIC_GetFaultAddress	返回产生表示出错的系统Handler所在位置的地址

EXTI函数库

函数名称	功能
EXTI_DeInit	将外设EXTI寄存器重设为默认值
EXTI_Init	根据EXTI_InitStruct中指定的参数初始化外设EXTI寄存器
EXTI_StructInit	把EXTI_InitStruct中的每一个参数按默认值填入
EXTI_GenerateSWInterrupt	产生一个软件终端
EXTI_GetFlagStatus	检查指定的EXTI线路标志位设置与否
EXTI_ClearFlag	清除EXTI线路挂起标志位
EXTI_GetITStatus	检查指定的EXTI线路触发请求发生与否
EXTI_ClearITPendingBit	清除EXTI线路挂起位

`misc.h`

`misc.c`

1、中断优先级**分组**:

`void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup);`

涉及到的寄存器: `SCB_AIRCR`

```
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)
{
    assert_param(IS_NVIC_PRIORITY_GROUP(NVIC_PriorityGroup));
    SCB->AIRCR = AIRCR_VECTKEY_MASK | NVIC_PriorityGroup;
}
```

例: `NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);`

2、中断优先级设置

```
void NVIC_Init(NVIC_InitTypeDef* NVIC_InitStruct);
```

```
typedef struct
{
    __IO uint32_t ISER[8];
        uint32_t RESERVED0[24];
    __IO uint32_t ICER[8];
        uint32_t RSERVED1[24];
    __IO uint32_t ISPR[8];
        uint32_t RESERVED2[24];
    __IO uint32_t ICPR[8];
        uint32_t RESERVED3[24];
    __IO uint32_t IABR[8];
        uint32_t RESERVED4[56];
    __IO uint8_t IP[240];
        uint32_t RESERVED5[644];
    __O uint32_t STIR;
} NVIC_Type;
```

涉及到的寄存器：

```
__IO uint8_t IP[240]; //中断优先级控制的寄存器组
__IO uint32_t ISER[8]; //中断使能寄存器组
__IO uint32_t ICER[8]; //中断失能寄存器组
__IO uint32_t ISPR[8]; //中断挂起寄存器组
__IO uint32_t ICPR[8]; //中断解挂寄存器组
__IO uint32_t IABR[8]; //中断激活标志位寄存器组
```

2、中断优先级设置

```
void NVIC_Init(NVIC_InitTypeDef* NVIC_InitStruct);
```

```
typedef struct
{
    uint8_t NVIC_IRQChannel; //设置中断通道
    uint8_t NVIC_IRQChannelPreemptionPriority; //设置抢占优先级
    uint8_t NVIC_IRQChannelSubPriority; //设置响应优先级
    FunctionalState NVIC_IRQChannelCmd; //使能/禁止
} NVIC_InitTypeDef;
```

```
NVIC_InitTypeDef  NVIC_InitStructure;
NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn; // 37 串口1中断
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=1; // 抢占优先级为1
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2; //响应优先级为2
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ通道使能
NVIC_Init(&NVIC_InitStructure); //根据上面指定的参数初始化NVIC寄存器
```

中断优先级设置步骤

① 系统运行后先设置中断优先级分组。调用函数：

```
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup);
```

整个系统执行过程中，只设置一次中断分组。

② 针对每个中断，设置对应的抢占优先级和响应优先级：

```
void NVIC_Init(NVIC_InitTypeDef* NVIC_InitStruct);
```

③ 如果需要挂起/解挂，查看中断当前激活状态，分别调用相关函数即可。用的不多！

通常偏上外设中断管理只需要第1步和第2步！

外部中断常用库函数

外部中断初始化程序

- ① 使能GPIO口的时钟、使能SYSCFG时钟：
`RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);`
- ② 初始化IO口为输入。
`GPIO_Init();`
- ③ 设置IO口与中断线的映射关系。
`void SYSCFG_EXTILineConfig();`
- ④ 初始化线上中断，设置触发条件等。
`EXTI_Init();`
- ⑤ 配置中断分组（NVIC），并使能中断。
`NVIC_Init();`

中断服务程序

- ⑥ 编写中断服务函数。
`EXTIx_IRQHandler();`
响应中断后在中断服务程序中要先判断中断来源，
`EXTI_GetITStatus();`
然后响应中断后在中断服务程序中要 清除中断标志位
`EXTI_ClearITPendingBit();`
- ⑦ 编写中断服务程序处理内容。

简单的可以在
`EXTIx_IRQH
andler();`
内实现，复杂
的设置标志位
后在主程序实
现。

外部中断常用库函数

stm32f4xx_exti.h
stm32f4xx_exti.c

```
62 | #define EXTI_PortSourceGPIOE      ((uint8_t) 0x04)
    |                                     ^
//连接GPIO端口引脚到中断源
1、 void SYSCFG_EXTILineConfig(uint8_t EXTI_PortSourceGPIOx, uint8_t
EXTI_PinSourcex);
    |                                     ^
92 | #define EXTI_PinSource2            ((uint8_t) 0x02)
    |                                     ^
```

例: `SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOE, EXTI_PinSource2);`

//初始化中断线: 选择中断源、中断模式、触发方式、使能等

2、 `void EXTI_Init(EXTI_InitTypeDef* EXTI_InitStruct);`

//判断中断线中断状态, 是否发生

3、 `ITStatus EXTI_GetITStatus(uint32_t EXTI_Line);`

//清除中断线上的中断标志位

4、 `void EXTI_ClearITPendingBit(uint32_t EXTI_Line);`

//这个函数非常重要, 在使用外部中断的时候一定要先使能SYSCFG时钟//设置IO口与中断线的映射关系

5、 `RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);` //使能SYSCFG时钟

STM32中断系统库函数

```
void EXTI_Init(EXTI_InitTypeDef* EXTI_InitStruct);
```

```
typedef struct
```

```
{
```

```
uint32_t EXTI_Line;    //指定要配置的中断线
```

```
EXTIMode_TypeDef EXTI_Mode;    //模式: 事件 OR中断
```

```
EXTITrigger_TypeDef EXTI_Trigger; //触发方式: 上升沿/下降  
沿/双沿触发
```

```
FunctionalState EXTI_LineCmd;    //使能 OR失能
```

```
}EXTI_InitTypeDef;
```

```
105 #define EXTI_Line0    ((uint32_t)0x00001)
```

```
106 #define EXTI_Line1    ((uint32_t)0x00002)
```

```
107 #define EXTI_Line2    ((uint32_t)0x00004)
```

Stm32f4xx_exti.h

例:

```
EXTI_InitStructure.EXTI_Line =EXTI_Line2;  
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;  
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;  
EXTI_InitStructure.EXTI_LineCmd = ENABLE;  
EXTI_Init(&EXTI_InitStructure);
```

```
66 typedef enum
```

```
67 {
```

```
68     EXTI_Trigger_Rising = 0x08,
```

```
69     EXTI_Trigger_Falling = 0x0C,
```

```
70     EXTI_Trigger_Rising_Falling = 0x10
```

```
71 }EXTITrigger_TypeDef;
```

```
54 typedef enum
```

```
55 {
```

```
56     EXTI_Mode_Interrupt = 0x00,
```

```
57     EXTI_Mode_Event = 0x04
```

```
58 }EXTIMode_TypeDef;
```

04 应用实例

编写NVIC中断初始化程序实现如下功能：

- 1、设置中断优先级组为1组
- 2、设置外部中断1的**抢占优先级为0**，**响应优先级为2**
- 3、设置定时器1的溢出更新中断的**抢占优先级为1**，**响应优先级为4**
- 4、设置USART1的**抢占优先级为1**，**响应优先级为5**

并说明当同时出现以上3个中断请求时，中断服务程序执行的顺序？

1 NVIC中断初始化

```
void NVIC_Configuration(void)
{
    NVIC_InitTypeDef  NVIC_InitStructure;//声明NVIC初始化临时变量
    /* 将NVIC中断优先级组设置为1组*/ ①
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1); //1位抢占优先级可以是：0和1
                                                    //3位响应优先级可以是：0-7

    /* 1设置中断源 为EXTI1_IRQn */ ②-1
    NVIC_InitStructure.NVIC_IRQChannel = EXTI1_IRQn;
    /* 设置抢先优先级*/
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    /*设置响应优先级*/
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2;
    /* 使能这一中断 */
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    /* 完成以上中断的初始化功能*/
    NVIC_Init(&NVIC_InitStructure);
}
```

中断号：定义在stm32f4xx.h

199	EXTI1_IRQn	= 7,
-----	------------	------

1 NVIC中断初始化

```
/* 2设置中断源 为TIM1_UP_TIM10_IRQn */ ②-2
NVIC_InitStructure.NVIC_IRQChannel = TIM1_UP_TIM10_IRQn;
/* 设置抢占优先级*/
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
/*设置响应优先级*/
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 4;
/* 使能这一中断 */
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
/* 完成以上中断的初始化功能*/
NVIC_Init(&NVIC_InitStructure);

/* 3设置中断源 为USART1_IRQn*/ ②-3
NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
/* 设置抢占优先级*/
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
/*设置响应优先级*/
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 5;
/* 使能这一中断 */
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
/* 完成以上中断的初始化功能*/
NVIC_Init(&NVIC_InitStructure);
} //初始化函数结束
```

2 外部中断应用

要求:

按照图中的控制电路, 编写程序将PA0配置为外部中断输入, 将中断输入触发方式配置为上升沿触发, 并编写中断服务程序, 实现LED灯的控制。检测到按键动作将LED灯状态反转(忽略去抖动操作)。

1. 编程要点

(1) 使能GPIOA时钟和SYSCFG时钟。调用函数:

```
RCC_AHB1PeriphClockCmd();
```

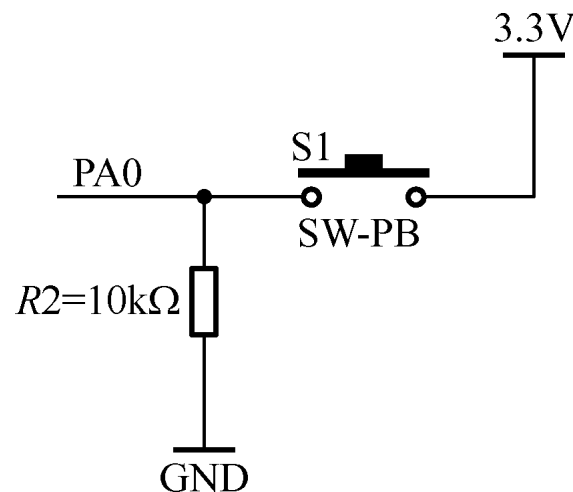
(2) 初始化PA0为输入模式。

(3) 设置EXTI线0的工作模式、触发方式和使能。

(4) 设置NVIC中断组和EXTI线0中断通道的优先级和使能状态。

(5) 编写中断服务程序。需要判断中断线状态, 在确认后, 清除中断挂起标志。 添加一个函数

```
RCC_APB2PeriphClockCmd();
```



外部中断触发电路图

2. 主程序

```
int main(void)
{
    /* LED 端口初始化 */
    LED_Config();
    /* 初始化EXTI中断, 按下按键会触发中断,
     * 触发中断会进入stm32f4xx_it.c文件中的函数, 反转LED灯。
     */
    EXTI_Key_Config();
    /* 等待中断, 由于使用中断方式, 不用扫描按键 */
    while(1)
    {
    }
}
```

在函数中，首先初始化LED灯的GPIO引脚，然后初始化EXTI中断相关内容，**最后在while(1)的空循环中等待中断。**

3. EXTI中断初始化函数

```
void EXTI_Key_Config(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    EXTI_InitTypeDef EXTI_InitStructure;
    //-----第一步-----
    /*开启按键GPIO口的时钟*/ ①
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA ,ENABLE);
    /* 使能 SYSCFG 时钟 , 使用GPIO外部中断时必须使能SYSCFG时钟*/
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
    //-----第二步-----
    /* 选择按键1的引脚 */ ②
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
    /* 设置引脚为输入模式 */
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
    /* 设置引脚不上拉也不下拉 */
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
    /* 使用上面的结构体初始化按键 */
    GPIO_Init(GPIOA, &GPIO_InitStructure);
    //-----第三步-----
    /* 连接 EXTI 中断源 到key1引脚 */ ③
    SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOA,EXTI_PinSource0);
```

3. EXTI中断初始化函数

```
//-----第四步-----
/* 选择 EXTI 中断源 */
EXTI_InitStructure.EXTI_Line = EXTI_Line0;
/* 中断模式 */
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
/* 上升沿触发 */
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
/* 使能中断/事件线 */
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
EXTI_Init(&EXTI_InitStructure);
//-----第五步-----
/*配置NVIC为优先级组1, 保证整个程序使用的中断处于同一组*/
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
/*配置中断源: 按键1*/
NVIC_InitStructure.NVIC_IRQChannel = EXTI0_IRQn;
/*配置抢占优先级: 1*/
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=1;
/*配置响应优先级: 1*/
NVIC_InitStructure.NVIC_IRQChannelSubPriority=1;
/*使能中断通道*/
NVIC_InitStructure.NVIC_IRQChannelCmd=ENABLE;
NVIC_Init(&NVIC_InitStructure);
//-----其他的外部中断 配置过程一样-----
} //初始化结束
```

4. 中断服务程序

```
void EXTI0_IRQHandler(void)
{
    //确保是否产生了EXTI Line中断
    if (EXTI_GetITStatus(EXTI_Line0) != RESET)
    {
        //清除中断标志位，否则难以响应后面的中断
        EXTI_ClearITPendingBit(EXTI_Line0);
        // LED 取反
        LED_TOGGLE;
    }
}
```

中断服务程序的名称一定要
和启动文件
startup_stm32f429_439
xx.s中定义的保持一致

⑥

⑦ 响应中断的目的

实现功能比较简单，直
接在服务程序内实现

中断服务程序实在响应中断后，自动调用的，可以放置于工程的任何地方。

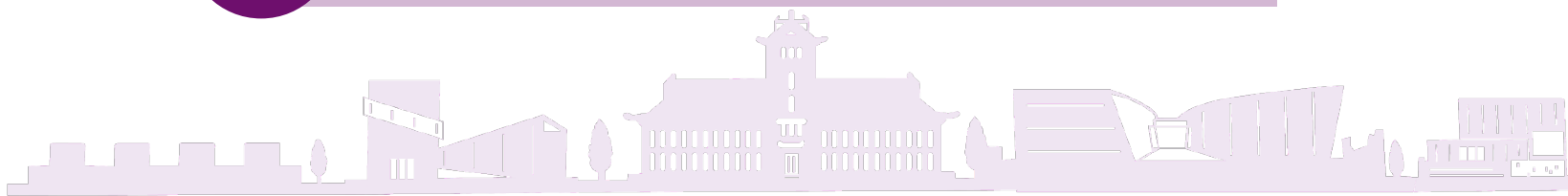
本章小结

01 中断的基本概念

02 STM32中断系统

03 STM32中断系统库函数

04 应用实例



谢谢!